

PROJECT ROADMAP

AI/ML-Based Object Detection System for Dual-Energy X-Ray Luggage Scanners

Complete Development Strategy · Dual-Model Architecture · Challenge Mitigation Guide

B.Tech Computer Science Engineering | VIT Pune | 2025–2026

0. Document Overview

This document provides the complete, step-by-step execution strategy for building the AI/ML-Based Object Detection System for Dual-Energy X-Ray Luggage Scanners. The project proposes an innovative two-model architecture: Model 1 extracts structured physical and material properties from X-ray images, and Model 2 classifies objects based on those extracted properties. This decoupled pipeline is fundamentally different from conventional end-to-end object detection, and that architectural distinction introduces a unique set of engineering and research challenges that this roadmap directly addresses.

This roadmap is organized into seven sequential phases, each with clearly defined objectives, deliverables, tools, sub-tasks, and — most importantly — a dedicated section documenting every anticipated challenge along with concrete mitigation strategies.

- Phase 0: Foundation Setup — environment, tools, and domain knowledge
- Phase 1: Dataset Strategy — data collection, Threat Image Projection, and augmentation
- Phase 2: Preprocessing Pipeline — normalization, enhancement, and segmentation
- Phase 3: Model 1 — Property Extraction Model design and training
- Phase 4: Model 2 — Property-Based Identification Model design and training
- Phase 5: System Integration — assembling the full dual-model pipeline
- Phase 6: Evaluation, Testing, and Benchmarking
- Phase 7: Optimization, Deployment, and Research Paper

Total projected duration: 20–24 weeks (part-time student team). Full-time teams: 12–14 weeks.

1. Recommended Approach: Why Two Models?

1.1 The Core Architectural Philosophy

Standard object detection systems like YOLO or Faster R-CNN operate end-to-end — they receive a raw image and directly output class labels and bounding boxes. These systems are powerful, but have a fundamental limitation: every class they detect must be explicitly represented in their training data. If a new threat object emerges, the model cannot recognize it without complete retraining.

The two-model property-based architecture breaks this dependency by introducing an intermediate representation layer — the property vector. Instead of training a model to recognize a knife directly, Model 1 learns to recognize properties like 'high-density metallic material, sharp linear edge structure, high length-to-width ratio.' Model 2 then maps that property description to the classification 'knife.' When a new threat emerges — say, a ceramic knife with identical shape but organic material — only Model 2's property-to-class mapping needs updating, without touching Model 1 at all.

1.2 Approach Comparison Table

Criterion	End-to-End CNN	YOLO Detection	Proposed Two-Model
Scalability to new threats	Poor — full retrain	Poor — new labels + retrain	Excellent — add property pattern only
Interpretability	None — black box	Low — bbox + label only	High — structured property explanation
Dataset requirement	Very high per class	High per class	Lower — extractor trained once
Dual-energy material use	Partial	Partial	Full — material is first-class input
Research novelty	Low	Moderate	High — underexplored paradigm

PHASE 0 Foundation Setup (Weeks 1–2)

Objective

Establish the full technical and conceptual foundation before writing any model code. Gaps in environment setup or domain knowledge here are the most common cause of project delays later.

► Domain Knowledge to Acquire

- Physics of dual-energy X-ray: how HE and LE beams produce material-differentiated pseudo-color images. Read Deshpande et al. (BARC 2022) and Mery et al. (IEEE Access 2020) fully.
- Beer-Lambert Law and Threat Image Projection (TIP) — critical for dataset construction.
- Color mapping in X-ray pseudo-images: orange = organic, blue/dark = metallic, green/grey = mixed.

► Deep Learning Foundations

- CNN architecture: convolution, pooling, batch normalization, dropout, fully connected layers.
- Detection vs. classification vs. segmentation — understand when each applies.

- Feature vectors and embeddings: what they represent and how to extract them from intermediate CNN layers.
- Multi-task learning: training one network with multiple loss objectives simultaneously.
- Transfer learning and fine-tuning from pre-trained ImageNet weights.

► Environment Setup Checklist

1. Install Python 3.10+ with Anaconda or virtualenv.
2. Install PyTorch with CUDA support. Verify GPU detection with `torch.cuda.is_available()`.
3. Install: torchvision, OpenCV, scikit-learn, scikit-image, numpy, pandas, matplotlib, albumentations.
4. Install: ultralytics (YOLOv8), detectron2 (Mask R-CNN for segmentation).
5. Create project directory structure: /data, /preprocessing, /model1, /model2, /pipeline, /evaluation, /docs.
6. Set up Git repository with GitHub/GitLab. Commit after every milestone.
7. Sign up for Weights & Biases (wandb.ai) for experiment tracking — free tier is sufficient.

Hardware Recommendation

A GPU with at least 6GB VRAM (NVIDIA GTX 1060 or above) is strongly recommended for Model 1 training. If no local GPU is available, use Google Colab Pro or Kaggle Notebooks (free GPU). CPU-only training is feasible for Model 2 and small experiments but prohibitively slow for Model 1.

► Phase 0 Deliverables

- Fully configured Python environment with all dependencies verified.
- Project directory pushed to version control.
- Reading notes on X-ray dual-energy imaging physics.
- One-page architecture sketch of the proposed system showing the full pipeline flow.

PHASE 1 Dataset Strategy & Construction (Weeks 3–5)

Objective

Model 1 needs images with pixel-level ground truth for object regions AND property labels. Model 2 needs structured feature vectors with class labels. These are fundamentally different data requirements — both must be planned and prepared in this phase.

Datasets to Acquire

► Public Benchmark Datasets (Start Here)

- SIXray Dataset (Miao et al., CVPR 2019): 1.06 million images, 6 threat categories. Use the SIXray10 or SIXray100 subsets initially to manage size.
- OPIXray Dataset (Wei et al., ACM Multimedia 2020): 8,885 X-ray images of cutting instruments across 5 sub-categories. Includes heavy occlusion scenarios — ideal for testing Model 1.

- HiXray Dataset: 45,364 images from real airport security checks. Closest to real operational conditions.

► **Simulated Data via Threat Image Projection (TIP)**

If public datasets lack dual-energy pseudo-color images, generate synthetic dual-energy training data using the TIP methodology with Beer-Lambert law computation. Even 20–30 clean dual-energy bag images from a single scanner session are sufficient as a TIP base set. Apply class-preserving translation and rotation augmentation (0° to 360° in 5° steps). Target: generate at least 5,000 augmented threat-positive images per threat category.

 **CRITICAL CHALLENGE — Severe Dataset Class Imbalance**

In SIXray, less than 1% of images contain threat items. A model trained on raw data will predict 'non-threat' for every image and still achieve 99% accuracy — but with catastrophically high false-negative rates in real deployment.

 **Mitigation: TIP + WeightedRandomSampler + Focal Loss**

Use TIP augmentation to achieve at minimum a 1:3 threat-to-non-threat ratio. Use PyTorch's WeightedRandomSampler to oversample the minority class during training. Use Focal Loss (gamma=2.0, alpha=0.25) instead of standard cross-entropy — it down-weights easy examples and forces the model to focus on hard threat examples.

Property Label Annotation Strategy

This is the most critical and most overlooked step in the project. Model 1 outputs property vectors, not class labels. Your training data therefore needs property-level annotations. Define exactly what 10 properties your system will extract, then annotate your training images with those properties.

Property	Type	Value Range	Extraction Source
Edge Sharpness Score	Float	0.0 – 1.0	Canny edge detection on object mask
Length-to-Width Ratio	Float	0.0 – 10.0+	Bounding box dimensions
Symmetry Score	Float	0.0 – 1.0	Axis-aligned pixel distribution comparison
Curvature Index	Float	0.0 – 1.0	Contour curvature analysis
Approximate Volume	Float (normalized)	Normalized	Pixel area × opacity-estimated depth
Material Category	Categorical (4 classes)	Organic/Metallic/Mixed/Opaque	Dual-energy HE/LE channel analysis
Average Absorption Intensity	Float	0.0 – 1.0	Mean pixel value in grayscale channel
Material Homogeneity	Float	0.0 – 1.0	Std deviation of absorption across object
Density Level	Float	0.0 – 1.0	Weighted HE+LE intensity ratio

Sharp Edge Count	Integer	0 – 20	Harris / Shi-Tomasi corner detection
------------------	---------	--------	--------------------------------------

⚠ CRITICAL CHALLENGE — Property Annotation Bottleneck

Manually annotating thousands of images with 10-dimensional property vectors is extremely time-consuming. If relying entirely on human annotators, this phase can delay the entire project by weeks.

✓ Mitigation: Semi-Automated property_annotator.py Script

Build a Python script that takes an annotated X-ray image (bounding boxes and material category from LabelMe/CVAT) and automatically computes all remaining 9 property values using OpenCV and NumPy. Human annotators only label bounding boxes and material category — everything else is computed automatically. Reduces annotation time from 5–10 minutes to under 30 seconds per image.

► **Phase 1 Deliverables**

- At least 5,000 annotated training images per threat class (via TIP augmentation) with bounding boxes, segmentation masks, and property vector labels.
- property_annotator.py script for semi-automated property computation.
- Data manifest CSV: every training image listed with its property vector and class label.
- Train/validation/test split: 70% / 15% / 15% with class-balanced distribution in each split.

PHASE 2 Preprocessing Pipeline (Weeks 4–6)

Objective

Transform raw X-ray images into a standardized, enhanced format that maximizes performance for both models. For dual-energy X-ray, preprocessing is more complex than for standard RGB images because HE and LE channels must be handled separately and then fused strategically.

Step-by-Step Preprocessing Pipeline

► **Step 1: Four-Channel Tensor Construction**

Do not treat the dual-energy image as a simple RGB image. Construct a four-channel tensor: (-log HE, -log LE, HE+LE, HE-LE). The log transformation linearizes Beer-Lambert attenuation. The sum channel captures total density; the difference channel captures material-specific differential absorption — the most discriminative signal for material classification.

► **Step 2: Noise Reduction**

Apply a bilateral filter (cv2.bilateralFilter, d=9, sigmaColor=75, sigmaSpace=75) — NOT a Gaussian blur. The bilateral filter preserves sharp edges while smoothing flat regions. Edge preservation is critical because edge sharpness is one of your 10 property dimensions, and blurring it will corrupt your ground truth.

► **Step 3: CLAHE Normalization**

Normalize each channel independently to [0,1]. Then apply CLAHE (cv2.createCLAHE, clipLimit=2.0, tileGridSize=(8,8)) to each channel. CLAHE improves visibility of low-contrast objects buried under other items without over-amplifying noise in uniform regions.

► Step 4: Letterbox Resize

Resize all images to 640×640 pixels using letterbox padding (adding neutral borders), NOT stretch resizing. Distorting aspect ratio corrupts the length-to-width ratio property that Model 1 is supposed to extract accurately.

► Step 5: Instance Segmentation

Use Mask R-CNN (detectron2) fine-tuned on your X-ray dataset to generate instance segmentation masks. These masks define the pixel regions over which all shape and material properties are subsequently computed. This step is unique to the property-based pipeline — conventional YOLO pipelines do not need it.

⚠ CRITICAL CHALLENGE — Segmentation Quality Under Occlusion

Inaccurate segmentation masks that include pixels from adjacent overlapping objects will corrupt every property vector computed from those masks. This is the most technically demanding step in preprocessing.

✓ Mitigation: Three-Stage Segmentation Refinement

Use a three-stage pipeline: (1) Deep saliency detection (U2-Net) to identify foreground regions. (2) Mask R-CNN for coarse instance masks. (3) GrabCut (cv2.grabCut) initialized with the Mask R-CNN bounding boxes for mask refinement. This combination significantly outperforms single-stage Mask R-CNN on cluttered X-ray images.

► Augmentation During Training

- Geometric: random horizontal flip, rotation ($\pm 30^\circ$), scale (0.8–1.2×), elastic deformation.
- Photometric: brightness/contrast adjustment ($\pm 15\%$), Gaussian noise injection, random gamma correction.
- X-ray specific: simulate occlusion with Cutmix-style patch pasting from other images.
- Do NOT apply color jitter to the material channel — the pseudo-color encoding carries semantic material information that must not be randomly altered.

► Phase 2 Deliverables

- preprocessing.py module with all five preprocessing steps as modular, testable functions.
- Preprocessing benchmark: average processing time < 500ms per image (training), < 100ms (inference).
- Visual inspection report: 100 sample preprocessed images reviewed for quality.

PHASE 3 Model 1 — Property Extraction Model (Weeks 6–11)

Objective

Model 1 takes a preprocessed X-ray image and produces two outputs: (1) object detection — bounding boxes and segmentation masks for suspicious objects; and (2) property extraction — the 10-dimensional structured property vector for each detected object. This is a multi-task learning model, which is both its strength and the source of its most significant engineering challenges.

Recommended Architecture: Modified YOLOv8 with Property Regression Head

Use YOLOv8 (Ultralytics) as the backbone detection network, extended with a secondary property regression head running in parallel to the standard detection head.

► Architecture Components

- Backbone: CSPDarknet53 — hierarchical feature extraction from the four-channel input tensor.
- Neck: PANet (Path Aggregation Network) — multi-scale feature fusion for detecting both small objects (cartridges, needles) and large objects (laptops).
- Head 1 (Standard Detection Head): Predicts bounding boxes and threat vs. non-threat classification.
- Head 2 (Property Regression Head): A 3-layer MLP (512→256→10) with ReLU activations and batch normalization, branching from the same feature map as Head 1. Regresses the full 10-dimensional property vector.
- Material Branch (Optional): Dedicated shallow CNN branch on the HE-LE difference channel for 4-class material category prediction (organic/metallic/mixed/opaque).

► Training Strategy — Two Stages

8. Stage 1 (Weeks 6–8): Pre-train the full YOLOv8 backbone using standard detection loss only (bbox + objectness + class). Initialize from pre-trained COCO weights. Train for 50–100 epochs until detection stabilizes. This ensures the backbone has strong visual feature representations before property learning begins.
9. Stage 2 (Weeks 9–11): Freeze the backbone weights. Train only the Property Regression Head and Material Branch for 30–50 epochs using MSE loss for continuous properties and cross-entropy for material category. Staged training prevents property regression from destabilizing the backbone.

⚠ CRITICAL CHALLENGE — Multi-Task Gradient Conflict

When training detection and property regression simultaneously, detection loss gradients (typically larger magnitude) dominate, causing the network to largely ignore property learning. This is called gradient dominance and is the most common failure mode in multi-task deep learning.

✓ Mitigation: Uncertainty Weighting + Staged Training + Gradient Clipping

Implement Uncertainty Weighting (Kendall et al., 2018): each task has a learnable uncertainty parameter that automatically scales its loss contribution, preventing any single task from dominating. Combined with staged training (detection first) and gradient clipping (max_norm=1.0) to prevent exploding gradients, this strategy reliably produces well-balanced multi-task training.

⚠️ CRITICAL CHALLENGE — Noisy Property Labels Corrupting Training

Property labels computed programmatically (e.g., edge sharpness via Canny) are sensitive to preprocessing parameters. Inconsistent preprocessing produces noisy labels that fundamentally limit Model 1's ceiling performance.

✓ Mitigation: Pre-Training Property Label Quality Audit

Before starting Model 1 training, plot the distribution of every property dimension across the full training set. Verify distributions match domain expectations (metallic objects should cluster at high absorption intensity; organic objects at lower values). Flag and inspect all outliers. Correct or remove them before training begins. This upfront audit prevents 'garbage in, garbage out' dynamics.

⚠️ CRITICAL CHALLENGE — Occlusion-Induced Property Corruption

When objects overlap, segmentation masks may include pixels from adjacent objects. The resulting property vector becomes a blend of two objects' properties rather than an accurate description of either.

✓ Mitigation: Occlusion Score as 11th Property Dimension

Add an occlusion score (computed as the fraction of bounding box covered by overlapping objects) as the 11th dimension of the property vector. Train Model 2 to treat highly occluded objects (score > 0.4) as candidates for human review rather than automated classification. This makes the system honest about its uncertainty under adverse imaging conditions.

► Evaluation Metrics for Model 1

- Detection: mAP@0.5 and mAP@0.5:0.95. Target: mAP@0.5 > 90%.
- Property regression: MAE and RMSE per property dimension. Target: normalized MAE < 0.10.
- Material classification accuracy: > 92% on held-out validation set.
- Property vector consistency: cosine similarity between vectors of the same class should cluster visibly when plotted with t-SNE.

► Phase 3 Deliverables

- Fully trained Model 1 checkpoint saved in .pth format.
- Training loss curves for detection head and property regression head across all epochs.
- Property extraction demo script: takes an X-ray image, outputs annotated bounding boxes with printed property vectors.
- Evaluation report with mAP scores and per-property MAE values on the validation set.

PHASE 4 Model 2 — Property-Based Identification Model (Weeks 11–14)

Objective

Model 2 receives the structured property vector from Model 1 and maps it to a specific object class (gun, knife, battery, explosive) with a calibrated threat probability score. Unlike Model 1, which is a deep learning model trained on images, Model 2 operates entirely on structured numerical data — making it much simpler, faster, and more interpretable.

Recommended Algorithm Progression

► Option 1: Random Forest (Start Here)

Fast, requires minimal hyperparameter tuning, handles mixed feature types naturally, and produces feature importance scores that directly reveal which properties are most discriminative per threat class. Start with 100 trees, tune with cross-validation.

► Option 2: XGBoost (If Random Forest Is Insufficient)

Gradient boosting achieves higher accuracy than random forests on structured tabular data in most benchmarks. More sensitive to hyperparameters but well worth the tuning effort if Random Forest accuracy falls below threshold.

► Option 3: Lightweight MLP (If Non-Linearity Is Required)

A 3-layer MLP (11→64→32→num_classes) with ReLU activations and dropout (0.3) handles complex non-linear property boundaries while remaining an order of magnitude faster than full CNN inference.

⚠ CRITICAL CHALLENGE — Train-Test Distribution Shift (Most Dangerous Failure Mode)

Model 2 is trained on clean ground-truth property vectors, but at inference time receives noisy predicted vectors from Model 1. A Model 2 trained only on clean vectors will perform well offline but degrade significantly in end-to-end inference because it has never been exposed to Model 1's error distribution. This is the most subtle and dangerous failure mode in the entire dual-model pipeline.

✓ Mitigation: Noise-Augmented Training (Self-Training)

After Model 1 is trained, run it on your full training set and collect all its predicted property vectors. Use these predicted vectors — with their ground-truth class labels — as additional training data for Model 2. This 'self-training' approach exposes Model 2 to Model 1's real error distribution, significantly closing the train-test gap. Train Model 2 on a 50/50 mix of clean ground-truth vectors and Model 1-predicted vectors.

⚠ CRITICAL CHALLENGE — Confusable Benign Objects

Metallic rulers, nail files, and scissors have property profiles nearly identical to knife blades (high metal density, sharp edges, high length-to-width ratio). A model trained only on threat vs. non-threat binary labels without these confusable classes will generate excessive false alarms.

✓ Mitigation: Explicit Confusable Class Training + Calibrated Thresholds

Include confusable benign objects (rulers, nail files, metallic water bottle components) as explicit negative classes in Model 2 training data. Output calibrated probability scores — not hard class labels. Use operational thresholds: probability > 0.95 triggers automatic alert; 0.70–0.94 triggers secondary human review; < 0.70 passes as non-threat. This tiered alert system dramatically reduces operational false-positive burden.

⚠ CRITICAL CHALLENGE — Adding New Threat Categories Requires Retraining

The core promise of the two-model architecture is instant scalability to new threats. But in practice, Random Forest and MLP classifiers require full retraining when new classes are added. This threatens to undermine the scalability claim.

✓ Mitigation: Property Pattern Database with Rule-Based Override Layer

Maintain a separate Property Pattern Database (patterns.yaml) that defines min/max ranges for each property dimension per threat category. When Model 1 produces a property vector, run it against the Pattern Database first — this is the rule-based override layer. New threats can be added to the database immediately without any retraining. The trained Model 2 handles probabilistic nuanced discrimination between known classes; the Pattern Database handles instant integration of new threats.

► Interpretability Output

Implement an explanation generator that for every flagged object outputs a human-readable explanation listing which property dimensions triggered the alert, their actual values, and the thresholds that were exceeded. This transparency is a key differentiating feature of the proposed system.

► Phase 4 Deliverables

- Trained Model 2 classifier (sklearn .pkl or PyTorch .pth).
- Feature importance chart showing which property dimensions are most discriminative per class.
- Confusion matrix on validation set. Target: False Negative Rate < 5% on threat classes.
- Explanation generator module (explain_prediction.py) with human-readable property evidence summaries.
- Property Pattern Database (patterns.yaml) with manually defined property ranges per threat category.

PHASE 5 System Integration — Full Dual-Model Pipeline (Weeks 14–17)

Objective

Phase 5 assembles the two independently trained models into a single coherent inference pipeline, manages the communication interface between them, and validates the integrated system end-to-end. This is where the unique architectural risks of the dual-model approach materialize most concretely.

Full Pipeline Execution Flow

10. Input: Dual-energy X-ray image loaded as a four-channel tensor (HE, LE, HE+LE, HE-LE).
11. Preprocessing Module: Bilateral filter → CLAHE → normalization → letterbox resize.
12. Model 1 Inference: Modified YOLOv8 produces bounding boxes, segmentation masks, and predicted property vectors.
13. Property Vector Serialization: Format each object's properties as a validated PropertyVector object.
14. Pattern Database Check: Rule-based range matching against patterns.yaml for instant new-threat detection.
15. Model 2 Inference: Trained classifier produces class probability scores from the property vector.
16. Decision Fusion: Weighted combination of Pattern Database score (0.4) and Model 2 probability (0.6).
17. Alert Generation: Apply tiered thresholds (HIGH / REVIEW / CLEAR) with property explanation output.

18. Visualization: Output image with color-coded bounding boxes, threat level badges, and property summaries.

⚠️ CRITICAL CHALLENGE — Pipeline Latency Exceeding Real-Time Limit

Airport X-ray conveyors allow approximately 2.5 seconds per bag. The full dual-model pipeline — preprocessing + Model 1 + segmentation + property computation + Model 2 — adds significant overhead beyond single-model YOLO inference. Without optimization, CPU-only deployment will exceed the time limit.

✓ Mitigation: Async Pipeline + Tiny-YOLOv8 + Vectorized Property Computation

Implement asynchronous processing using Python asyncio or a producer-consumer queue: start preprocessing the next frame while Model 1 runs on the current frame. Vectorize all OpenCV property computations using NumPy batch operations. On CPU-only hardware, use Tiny-YOLOv8 for Model 1 (validated sub-0.4 second CPU inference). Target: < 1.5 seconds total pipeline latency on mid-range hardware.

⚠️ CRITICAL CHALLENGE — Property Vector Schema Mismatch Between Model 1 and Model 2

The scale and normalization of property vectors that Model 1 outputs must exactly match what Model 2 expects. Any inconsistency will silently corrupt Model 2's predictions without any obvious error — an extremely difficult bug to detect after deployment.

✓ Mitigation: Strict PropertyVector Class with Automated Schema Validation

Define a formal PropertyVector Python dataclass (or pydantic model) that enforces the exact schema: dimension order, value ranges, normalization method. Both Model 1's output serializer and Model 2's input deserializer must use this same class. Add an assert_valid() method that raises a ValueError if any dimension is out of range. Run this validation on every property vector during development — the overhead is negligible compared to the debugging time it saves.

Integration Test Protocol

19. Single threat image end-to-end test: verify correct class output and property values.
20. Single benign image test: verify CLEAR output with zero false positives.
21. Batch test on 100 images: measure average latency, peak memory, and verify no NaN property values.
22. Edge case — empty bag: verify graceful handling when Model 1 detects no objects.
23. Edge case — 5+ overlapping objects: verify all are individually detected and classified.
24. Occlusion stress test: verify occlusion score dimension correctly reflects degree of overlap.

► Phase 5 Deliverables

- pipeline.py module implementing the full end-to-end inference chain.
- PropertyVector class with schema validation and serialization.
- Integration test suite with pass/fail results for all 6 test categories.
- Pipeline latency benchmark: mean and 95th-percentile on 500 test images.
- demo.py script: processes a folder of test images and saves annotated outputs.

PHASE 6 Evaluation, Testing & Benchmarking (Weeks 17–20)

Objective

Conduct comprehensive, rigorous evaluation of the complete integrated system against standard benchmarks. The results here form the core experimental section of your research paper. All evaluation must be performed on a held-out test set never seen during training.

Evaluation Metrics

► Detection-Level (Model 1)

- mAP@0.5 and mAP@0.5:0.95 on SIXray and OPIXray test sets.
- Property Extraction RMSE: per-dimension RMSE between predicted and ground-truth property vectors.
- Material Classification Accuracy: > 92% target on held-out set.

► Classification-Level (Model 2)

- Per-class Precision, Recall, and F1 Score for each threat category.
- False Negative Rate (FNR): fraction of actual threats missed. Target: < 3%. This is the most critical metric.
- False Positive Rate (FPR): fraction of benign items incorrectly flagged. Target: < 5%.
- AUC-ROC for threat vs. non-threat binary decision.

► System-Level (End-to-End Pipeline)

- End-to-end inference time: mean and 95th-percentile latency on GPU and CPU hardware.
- Scalability validation: add 3 new threat categories via Pattern Database only (no retraining) and measure detection effectiveness — this is your strongest research contribution to demonstrate.
- Interpretability quality: user study with 5–10 participants (security personnel or domain experts) rating explanation usefulness on a 1–5 scale.

Comparison Baseline Requirement

Your research paper must demonstrate measurable advantages over existing approaches. Set up a standard YOLOv8 baseline trained identically on the same dataset. Compare FNR, FPR, and the scalability test. The scalability comparison — showing your system handles new threats without retraining while YOLO requires full retraining — is your most compelling differentiator.

► Phase 6 Deliverables

- Full evaluation report with all metrics for Model 1, Model 2, and the end-to-end pipeline.
- Comparison table: proposed system vs. YOLOv8 baseline vs. standard CNN classifier.
- Latency benchmark results on both GPU and CPU hardware.
- Scalability demonstration evidence with 3 new threat categories added via Pattern Database.
- Confusion matrix visualization for all threat classes.

PHASE 7 Optimization, Documentation & Research Paper (Weeks 20–24)

Model Optimization

► Post-Training Quantization

Apply PyTorch's `torch.quantization` to convert Model 1 float32 weights to int8, reducing model size approximately 4× and improving inference speed by 1.5–2×. Validate that quantized performance does not degrade by more than 2% mAP.

► ONNX Export and Deployment

Export both Model 1 and Model 2 to ONNX format for framework-agnostic deployment. Use ONNX Runtime for inference, providing optimized execution on both CPU and GPU across all operating systems. Test ONNX pipeline output parity against the original PyTorch pipeline.

Target Venues for Research Paper Publication

- IEEE International Conference on Image Processing (ICIP) — top venue for image processing research.
- MDPI Sensors (open-access) — strong track record of X-ray security papers; faster review cycle.
- Pattern Recognition Letters (Elsevier) — highly cited, moderate review turnaround.
- IEEE Transactions on Information Forensics and Security — premium journal for security AI applications.

Paper Structure

25. Abstract (250 words): problem, approach, key results, significance.
26. Introduction: motivate scalability and interpretability limitations of existing systems; state contributions explicitly.
27. Related Work: CNN classifiers, YOLO detectors, property-based detection, TIP augmentation.
28. Proposed Architecture: full dual-model pipeline description with architecture diagram.
29. Dataset and Experimental Setup: data sources, augmentation, training protocol — fully reproducible details.
30. Results: all metrics, comparison tables, latency benchmarks, qualitative annotated image examples.
31. Discussion: honest analysis of where the system excels and where limitations exist — shows scientific maturity.
32. Conclusion and Future Work: contributions summary + extensions (3D CT, multi-view fusion, active learning).

► Phase 7 Deliverables

- Quantized ONNX-exported pipeline with verified performance parity.
- Complete GitHub repository with README, installation guide, usage examples, and dataset download scripts.
- Full technical project report with architecture diagrams, evaluation results, and code documentation.

- Research paper draft ready for supervisor review and submission.

8. Master Challenge & Mitigation Reference Table

Use this table as a quick-reference guide when you encounter difficulties during development. Every challenge in this table is discussed in detail in the relevant phase section above.

#	Challenge	Phase	Severity	Mitigation Summary
1	Dataset class imbalance (threat < 1% of images)	Phase 1	CRITICAL	TIP augmentation + WeightedRandomSampler + Focal Loss
2	Property annotation bottleneck	Phase 1	HIGH	Semi-automated property_annotator.py for programmatic label computation
3	Segmentation mask corruption under occlusion	Phase 2	CRITICAL	Three-stage: Saliency (U2-Net) → Mask R-CNN → GrabCut refinement
4	Multi-task gradient conflict in Model 1	Phase 3	HIGH	Uncertainty Weighting loss + staged training + gradient clipping (max_norm=1.0)
5	Noisy property labels corrupting Model 1	Phase 3	HIGH	Pre-training statistical audit of all property label distributions
6	Occlusion-corrupted property vectors	Phase 3	MEDIUM	Occlusion score as 11th property dimension; flag occluded objects for human review
7	Train-test distribution shift: M1→M2	Phase 4	CRITICAL	Noise-augmented / self-training: train Model 2 on Model 1's predicted vectors
8	Confusable benign objects (rulers, nail files)	Phase 4	HIGH	Include confusable classes in Model 2 + calibrated probability alert thresholds
9	New threat class forces full Model 2 retraining	Phase 4	MEDIUM	Property Pattern Database with rule-based override layer for instant new-threat support
10	Pipeline latency > 2.5 second real-time limit	Phase 5	HIGH	Async pipeline + Tiny-YOLOv8 + vectorized NumPy property computation
11	Property vector schema mismatch M1↔M2	Phase 5	CRITICAL	Strict PropertyVector dataclass with automated range validation at every boundary
12	High false negative rate on novel / occluded threats	Phase 6	CRITICAL	Ensemble Model 2 with Pattern Database; uncertain predictions routed to human review
13	Model too large for edge/embedded deployment	Phase 7	MEDIUM	Post-training int8 quantization + optional knowledge distillation to student model

9. Master Timeline

Phase	Title	Weeks	Key Milestone
-------	-------	-------	---------------

Phase 0	Foundation Setup	1–2	Environment configured, architecture sketch complete
Phase 1	Dataset Strategy	3–5	Annotated dataset with property vectors ready
Phase 2	Preprocessing Pipeline	4–6	Preprocessing module validated, segmentation masks verified
Phase 3	Model 1 — Property Extraction	6–11	Model 1 trained: mAP@0.5 > 90%, property MAE < 0.10
Phase 4	Model 2 — Identification	11–14	Model 2 trained: FNR < 5%, explanation generator working
Phase 5	System Integration	14–17	Full pipeline assembled, all integration tests passed
Phase 6	Evaluation & Benchmarking	17–20	Full evaluation report with comparison baselines complete
Phase 7	Optimization & Paper	20–24	ONNX export done, research paper draft submitted to venue

10. Recommended Technology Stack

Category	Tool / Library	Purpose
Language	Python 3.10+	All model development, preprocessing, and pipeline code
Deep Learning	PyTorch + torchvision	Model 1 architecture, training loop, and inference
Detection Backbone	Ultralytics YOLOv8	Base detection backbone extended with property regression head
Segmentation	detectron2 (Mask R-CNN)	Instance segmentation for property extraction isolation
Computer Vision	OpenCV (cv2)	Preprocessing, property computation, visualization
Image Augmentation	albumentations	Training-time augmentation pipeline
ML Classifiers	scikit-learn + XGBoost	Model 2 classifier options (Random Forest / XGBoost)
Data Management	pandas + NumPy	Property vector storage, manipulation, and statistics
Experiment Tracking	Weights & Biases (wandb)	Training curves, hyperparameter logging, model comparison
Model Deployment	ONNX + ONNX Runtime	Framework-agnostic export and optimized inference
Annotation Tool	LabelMe or CVAT	Bounding box and segmentation mask annotation
Version Control	Git + GitHub	Code versioning and team collaboration
Hardware (Preferred)	NVIDIA GPU (≥6GB VRAM)	Model 1 training acceleration
Hardware (Fallback)	Google Colab Pro / Kaggle	Free GPU access for cloud training

11. Final Strategic Advice

- **Start With a Minimum Viable Pipeline Early**

By the end of Week 10, have a rough end-to-end pipeline running — even if Model 1 only extracts 3 properties and Model 2 is a simple threshold rule. Early integration surfaces interface problems and latency issues before months are spent fine-tuning individual components.

► **The Property Schema Is Your Most Critical Design Decision**

The 10-property vector definition made in Phase 1 propagates through every subsequent phase. If you later discover a property dimension is too noisy or non-discriminative, you will need to retrain both models. Validate the schema with a 50-image, 3-class feasibility experiment before committing to full-scale training.

► **The Dual-Model Challenges Are Your Research Story**

Every challenge in this roadmap — especially the train-test distribution shift between Model 1 and Model 2, the noise-augmented training strategy, and the scalability demonstration — is a research contribution in itself. Present them openly in your paper as novel engineering contributions, because that is genuinely what they are.

► **Document Everything From Day One**

Log every experiment — including failed ones — with hyperparameters, results, and observations using Weights & Biases. Your paper will be significantly stronger if it includes ablation studies showing why design choices were made, and a well-maintained experiment log is the source of that evidence.

End of Project Roadmap

AI/ML-Based Dual-Model Property-Based Detection System for Dual-Energy X-Ray Luggage Scanners

VIT Pune · B.Tech Computer Science Engineering · 2025–2026